Lab IV

The switch and break statement in C++

Asad Ullah

Electrical Ai

Semester 2nd

Section B

Reg No: BF25NWELE0686

Learning objectives:

- ***The objectives of this lab session are:***

1. ***To understand and implement the switch statement in C++.***
2. ***To apply the break statement to control program execution.***
3. ***To use the continue statement inside loops.***
4. ***To differentiate between break and continue statements.***
5. ***To develop simple menu-driven programs using switch-case.***

Theory:

Switch Statement

The switch statement is a multi-way selection control structure used to execute one block of code among multiple alternatives based on the value of a single expression.

It is commonly used as an alternative to multiple if-else statements when checking a single variable against different constant values. **General Syntax:**

```
switch(expression)
{
   case constant1:
statements;
      break;

   case constant2:
statements;
      break;

   ...    default:
statements;
}
```

• Important Rules of Switch Statement

For correct implementation, the following rules must be followed:

1. The switch expression must evaluate to:

   - int
   - char

- Enumeration type
  (Floating-point types like float or double are not allowed.)

2. Case values must be:
   - Constant expressions.
   - Unique (no duplicate case values allowed).
3. The default case is optional but recommended for safe programming.
4. If break is not used, execution continues into the next case (this is called fall-through behavior).

What is break statement?

The **continue** statement is used inside loops. Whenever a continue statement is encountered inside a loop, control directly jumps to the beginning of the loop for next iteration, skipping the execution of statements inside loop's body for the current iteration.

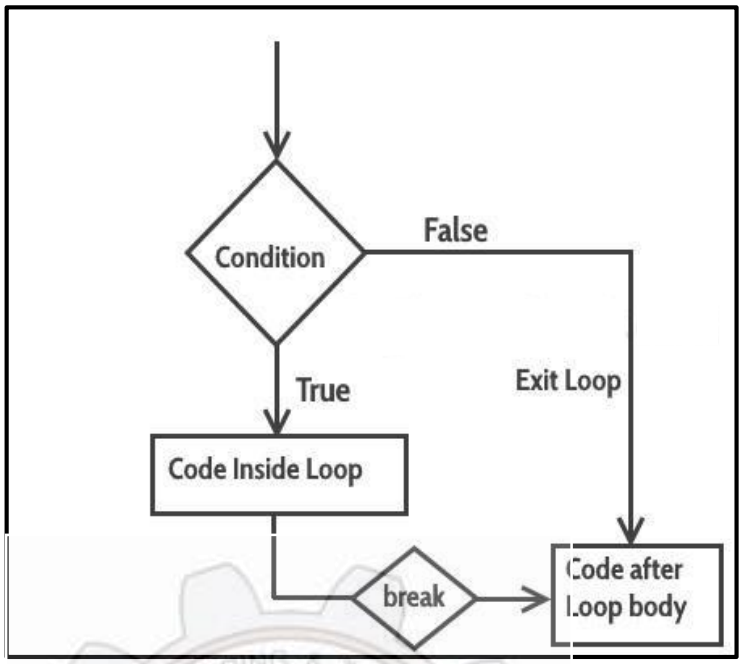
The **break statement** is used in following two scenarios:

- Use break statement to come out of the loop instantly. Whenever a break statement is encountered inside a loop, the control directly comes out of loop terminating it. It is used along with if statement, whenever used inside loop (see the example below) so that it occurs only for a particular condition.
- It is used in switch case control structure after the case blocks. Generally, all cases in switch case are followed by a break statement to avoid the subsequent cases (see the example below) execution. Whenever it is encountered in switch-case block, the control comes out of the switchcase body.

Why Break is Important?

Without break, the program continues executing the next cases even if they do not match. This may lead to incorrect results.

Flow chart of break statement:



Example: break statement in Switch Case.

```
#include <iostream> using
namespace std; int main(){    int
num=2;    switch (num) {      case
1: cout<<"Case 1 "<<endl;
break;      case 2: cout<<"Case 2
"<<endl;      break;      case 3:
cout<<"Case 3 "<<endl;      break;
     default: cout<<"Default "<<endl;
  }
  cout<<"Hey, I'm out of the switch case";
return 0;
}
```

Output:

```
Case 2
Hey, I'm out of the switch case
```

- In this example, we have break statement after each Case block, this is because if we don't have it then the subsequent case block would also execute.

  - The output of the same program without break would be:

```
Case 2
Case 3
Default
Hey, I'm out of the switch case
```

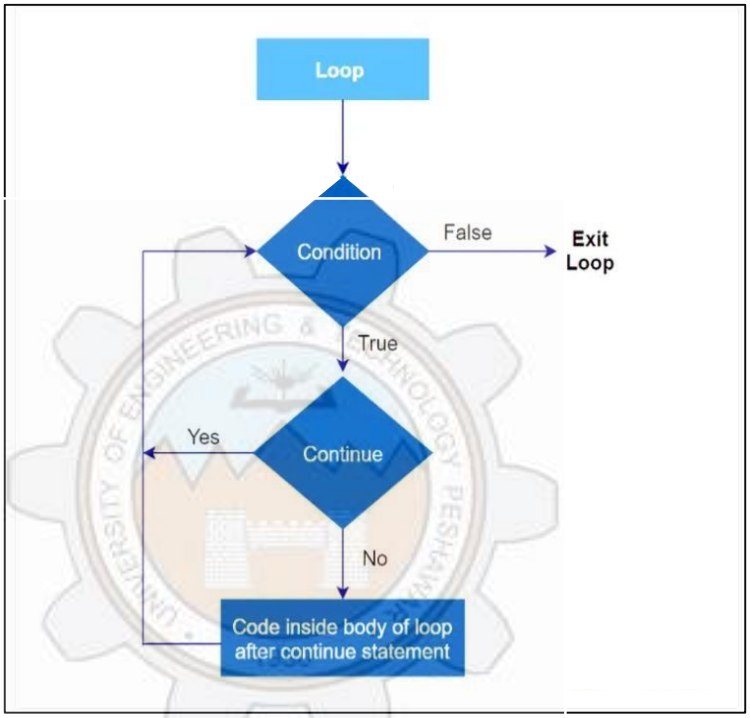
Example: Use of break statement in a while loop.

```
#include <iostream>
using namespace std;
int main () {    int var;
for (var =200;
var>=10; var --) {
cout<<"var:
"<<var<<endl;      if
(var==197) {
break;
    }
  }
  cout<<"Hey, I'm out
of the loop";    return
0;
}
```

What is continue statement?

The **continue** statement is used inside loops. Whenever a continue statement is encountered inside a loop, control directly jumps to the beginning of the loop for next iteration, skipping the execution of statements inside loop's body for the current iteration.

Flow chart of continue statement:



- **Example of continue statement with for loop in C++**

```
#include <iostream> using
namespace std;
int main ()
{
        for (int i = 1; i <= 10; i++)
          {
                if (i == 5)
                {
                continue;
                }
                cout << i << endl;
          }
 return 0;
}
```

- This C++ program iterates 10 times using a for loop.
- The continue statement is encountered when i=5. So, it skips the remainder of the loop's body for that iteration.
- As a result, it only prints the numbers 1-4 and 6-10, skipping 5.

Output:

```
1
2
3
4
6
7
8
9
10
```

Comparison of Break and Continue Statements in C++		
Parameters	**break statement in C ++**	**continue statement in C++**
Purpose	Exits the current loop	Skips the current iteration
Applicability	Used within loops (**for, while, do-while**)	Used within loops (**for, while, do-while**)
Effect	Terminates the loop prematurely and continues with the next statement after the loop	Immediately moves to the next iteration of the loop, skipping the remaining code in the current iteration
Loop Control	Affects the entire loop	Affects only the current iteration of the loop

Typical Use Cases	When a specific condition is met, and you want to exit the loop	When you want to skip the current iteration of the loop based on a certain condition and continue with the next iteration

Exercise:

1. Demonstrate continue statement with do...while loop in C++

Input:

```
#include <iostream> using
namespace std;
int main () {
int num = 1;
    do {        if (num % 2
== 0) {
            num++;
continue;
        }
        cout << num << " ";
num++;
    } while (num <= 10);
    cout << endl;
return 0;
}
```

Output:

```
1 3 5 7 9
```

2. Demonstrate break statement with while loop in C++

Input:

```
#include <iostream> using
namespace std;
int main() {     int num
= 1;     while (num <=
10) {        if (num ==
6) {
            break; }
cout << num << " ";
num++;
    }
    cout << endl;
return 0;
}
```

✞ **Output:**

```
1 2 3 4 5
```

Lab Tasks

- **Task 1:**
- **Write a C++ program that takes an input of a day's number and uses a switch statement to output**
- **the corresponding day's name. Here's how the program will proceed:**
- The program prompts the user to enter the day's number.
- The user enters a number between 1 and 7, where 1 is Monday and 7 is Sunday.

The program uses a switch statement to check which day was entered. Depending on the entered number, the corresponding day's name is outputted using the cout statement

Task 1: Day's Name Using Switch Statement

Problem Statement

- The program prompts the user to enter a number (1–7) representing a day of the week.
- Outputs the corresponding day name using a **switch statement**.
- Uses **break** to prevent fall-through.

Program

```
#include <iostream>
using namespace std;

int main() {
    int day;
    cout << "Enter a day number (1-7): ";
    cin >> day;

    switch(day) {
        case 1:
            cout << "Monday";
            break;
        case 2:
            cout << "Tuesday";
            break;
        case 3:
            cout << "Wednesday";
            break;
        case 4:
            cout << "Thursday";
            break;
        case 5:
            cout << "Friday";
            break;
        case 6:
            cout << "Saturday";
            break;
        case 7:
            cout << "Sunday";
            break;
        default:
            cout << "Invalid input";
    }

    return 0;
}
```

Explanation

- The user inputs a number 1–7.
- The switch statement checks which case matches.
- Each case prints the corresponding day and ends with **break** to prevent executing subsequent cases.
- The **default case** handles invalid input outside 1–7.

Task 2: Grade Evaluation Using Switch Statement

Problem Statement

- Input a **character grade** (A, B, C, D, F).
- Output the corresponding evaluation:

Grade	Result
A	Excellent
B	Very Good
C	Good
D	Pass
F	Fail

- Display "**Invalid Grade**" for any other character.

Program

```
#include <iostream>
using namespace std;

int main() {
    char grade;
    cout << "Enter your grade (A-F): ";
    cin >> grade;

    switch(grade) {
        case 'A':
            cout << "Excellent";
            break;
        case 'B':
            cout << "Very Good";
            break;
        case 'C':
            cout << "Good";
            break;
        case 'D':
            cout << "Pass";
            break;
        case 'F':
            cout << "Fail";
            break;
        default:
            cout << "Invalid Grade"; }
```

```
    return 0;}
```

Explanation

- Switch evaluates the **grade character**.
- Each case outputs the evaluation for the corresponding grade.
- **break** prevents execution of other cases.
- **default** handles invalid grades.

Task 3: Unit Conversion Menu Using Switch Statement

Problem Statement

- Display a menu of conversions:
  1 → Kilometers to Meters
  2 → Meters to Centimeters
  3 → Kilograms to Grams
  4 → Celsius to Fahrenheit
- Take user choice and input value.
- Perform the conversion and display result.
- Show "**Invalid choice**" if input is wrong.

Program

```
#include <iostream>
using namespace std;

int main() {
    int choice;
    float value;

    cout << "Conversion Menu:\n";
    cout << "1 -> Kilometers to Meters\n";
    cout << "2 -> Meters to Centimeters\n";
    cout << "3 -> Kilograms to Grams\n";
    cout << "4 -> Celsius to Fahrenheit\n";

    cout << "Enter your choice (1-4): ";
    cin >> choice;

    switch(choice) {
        case 1:
            cout << "Enter kilometers: ";
            cin >> value;
            cout << value << " km = " << value * 1000 << " meters";
            break;
        case 2:
            cout << "Enter meters: ";
            cin >> value;
```

```
            cout << value << " m = " << value * 100 << " cm";
            break;
        case 3:
            cout << "Enter kilograms: ";
            cin >> value;
            cout << value << " kg = " << value * 1000 << " grams";
            break;
        case 4:
            cout << "Enter Celsius: ";
            cin >> value;
            cout << value << "°C = " << (value * 9/5) + 32 << "°F";
            break;
        default:
            cout << "Invalid choice";
    }

    return 0;
}
```

Explanation

- Menu is displayed using **cout**.
- User enters **choice (1–4)**.
- Switch evaluates choice and performs the corresponding conversion.
- **break** prevents executing other cases.
- **default** handles invalid choices.

Conversions implemented:

- 1 km = 1000 m
- 1 m = 100 cm
- 1 kg = 1000 g
- Celsius → Fahrenheit: F=C×9/5+32F = C × 9/5 + 32F=C×9/5+32